

El efecto Mpemba cuántico en HPC

Modos lentos, evolución temporal, trayectorias y redes tensoriales

Juan | Sebastián

Sistemas Distribuidos / HPC — 2026

Implementación serial y paralela (OpenMP · MPI · CUDA), con validación y escalado

Hoja de ruta

1. De lo clásico (macro) a lo cuántico
2. T1 — Modos lentos del Liouvilliano
3. Evolución temporal (a): RK4 directo
4. Evolución temporal (b): Krylov
5. Muchos cuerpos: trayectorias (MPI)
6. Muchos cuerpos: redes tensoriales
7. Conclusiones generales
8. Relevancia en el efecto macro/meso

Juan modos lentos · redes tensoriales

Sebastián evolución temporal · trayectorias · GPU

1 · De lo clásico a lo cuántico

Juan

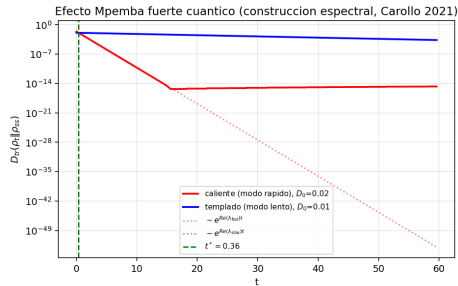
El efecto Mpemba: una anomalía de relajación

Juan

Clásico (macro). A veces el agua *más caliente* se congela *antes* que la templada: la curva que parte más lejos del equilibrio lo alcanza primero [16, 13, 10, 2].

Cuántico. En sistemas abiertos markovianos el fenómeno reaparece como un **cruce de las curvas de distancia** al estado estacionario [4, 17].

La firma universal: dos preparaciones, la que parte *lejos* adelanta a la que parte *cerca* en un tiempo t^* .



Cruce de curvas: el corazón del efecto.

Dinámica markoviana (Lindblad / GKSL) [12, 7, 3]:

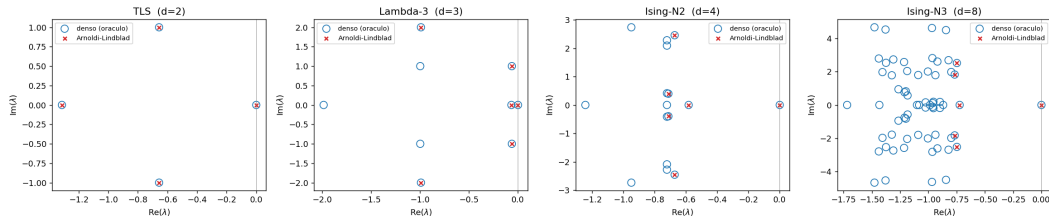
$$\dot{\rho} = \mathcal{L}[\rho] = -i[H, \rho] + \sum_{\mu} \left(L_{\mu} \rho L_{\mu}^{\dagger} - \frac{1}{2} \{ L_{\mu}^{\dagger} L_{\mu}, \rho \} \right).$$

Descomposición espectral del Liouvilliano (autovalores λ_k , $\text{Re } \lambda_k \leq 0$):

$$\rho_t = \rho_{\text{ss}} + \sum_{k \geq 2} a_k e^{\lambda_k t} \hat{r}_k, \quad a_k = \text{Tr}(\hat{l}_k^{\dagger} \rho_0).$$

- Relajación dominada por el **modo lento** λ_2 ($\tau = 1/|\text{Re } \lambda_2|$).
- **Criterio de Mpemba (fuerte)**: si $a_2 = \text{Tr}(\hat{l}_2^{\dagger} \rho_0) = 0$, la preparación *salta* el modo lento y relaja a la tasa rápida $\Rightarrow$ cruce [4, 5].
- No-normalidad de $\mathcal{L}$ ($\mathcal{L}\mathcal{L}^{\dagger} \neq \mathcal{L}^{\dagger}\mathcal{L}$): los modos no son ortogonales $\Rightarrow$ permite el adelantamiento.

T1: espectro de modos lentos -- Arnoldi-Lindblad vs diagonalización densa



Autovalores λ_k : $\lambda_1=0$ (estacionario ρ_{ss}), todos con $\text{Re } \lambda_k \leq 0$. El **modo lento** λ_2 —el más cercano a 0— fija la relajación y controla el cruce de Mpemba.

Dos preparaciones (estados producto puros), misma referencia ρ_{ss} (NESS): $\rho_A = |+\rangle^{\otimes N}$ (**parte LEJOS**) y $\rho_B = |0\rangle^{\otimes N}$ (**parte CERCA**). Con el espectro (T1: $\lambda_2, \hat{h}_2$) y los solapamientos $a_k = \text{Tr}(\hat{I}_k^\dagger \rho_0)$ (una traza, sin integrar):

estado	D_0 (arranque)	$ a_2 $ (peso modo lento $\lambda_2 = -0.70$)
$ +\rangle^{\otimes N}$ (lejos)	0.99	0.13
$ 0\rangle^{\otimes N}$ (cerca)	0.84	0.50

$|+\rangle$ arranca más alto pero pesa menos en el modo lento $\Rightarrow$ **cruce garantizado**, $t_{\text{pred}}^* \approx 0.68$ —
antes de evolucionar. T2/T3 lo confirman.

2 · T1 — Modos lentos del Liouvilliano

Juan

Objetivo. Extraer los autovalores de $\mathcal{L}$ más cercanos a 0 ($\lambda_2, \lambda_3, \dots$) y el solapamiento a_k , sin diagonalizar la matriz $4^N \times 4^N$.

Método: Arnoldi sobre el propagador $P(\tau) = e^{\tau\mathcal{L}}$ [15, 11].

- Acción *matrix-free* de P : se integra $\dot{V} = \mathcal{L}[V]$ con RK4 (no se forma $\mathcal{L}$).
- Como $\mu_k = e^{\tau\lambda_k}$ y $\text{Re } \lambda_k \leq 0$, los modos *lentos* de $\mathcal{L}$ son los *dominantes* de $P \Rightarrow$ Arnoldi los extrae primero (“*faster than the clock*”).

Paralelización. Arnoldi es secuencialmente acoplado $\Rightarrow$ el paralelismo vive en el **SpMV** (producto de matrices), no entre iteraciones.

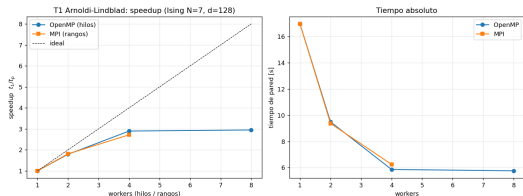
Serial

```
// producto C = A*B
for (int i=0;i<d;++i)
  for (int k=0;k<d;++k){
    cd a=A[i*d+k];
    for (int j=0;j<d;++j)
      C[i*d+j]+=a*B[k*d+j];
  }
```

Paralelo (datos)

```
#pragma omp parallel for          // intranodo
for (int i=rd.lo;i<rd.hi;++i)    // bloque del rango
  for (int k=0;k<d;++k){
    cd a=A[i*d+k];
    for (int j=0;j<d;++j)
      C[i*d+j]+=a*B[k*d+j];
  }
MPI_Allgatherv(...);           // reúne C (internodo)
```

Cada rango calcula un *bloque de filas* (OpenMP) y se reconstruye la matriz con **MPI_Allgatherv** (paralelismo de datos). El SpMV de Arnoldi llama a este núcleo.



Escalado fuerte (Ising $N = 7$, $d = 128$).

Computacional.

- OpenMP $2.9\times$ / MPI $2.7\times$ (4 workers).
- *Plateau* en los núcleos físicos: SpMV *memory-bound*; el $\mathcal{L}$ está **acoplado** $\Rightarrow$ comunicación Allgatherv.

Validación / física.

- Error en λ lentos: 10^{-13} – 10^{-5} vs denso.
- *Faster-than-the-clock* hasta $8.8\times$ en SpMV.
- Da el λ_2 y el a_2 que **determinan el cruce** de Mpemba.

3 · Evolución temporal (a): RK4 directo

Sebastián

Framework. Integrar $\dot{\rho} = \mathcal{L}[\rho]$ paso a paso desde una preparación ρ_0 y trazar $D_{HS}(\rho_t \| \rho_{ss})$.

Discretización: Runge–Kutta 4 (RK4), $\rho_{n+1} = \rho_n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$, con cada $k_i = \mathcal{L}[\cdot]$ *matrix-free* $\mathcal{L}[V] = -i(HV - VH) + \sum_{\mu} (L_{\mu} V L_{\mu}^{\dagger} - \frac{1}{2} \{L_{\mu}^{\dagger} L_{\mu}, V\})$.

- Orden global $\mathcal{O}(\Delta t^4)$; coste por paso $\mathcal{O}(Nd^3)$ (el **producto de matrices** densas domina).
- **Paralelización: OpenMP.** Pasos con dependencia temporal estricta $\Rightarrow$ el paralelismo está *dentro* del paso, en el `matmul`. Memoria compartida, sin halo $\Rightarrow$ OpenMP (no MPI).

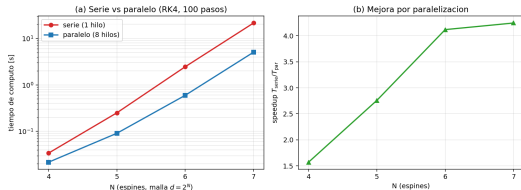
Serial

```
Mat matmul(A,B,d){
  Mat C(d*d);
  for(int i=0;i<d;++i)
    for(int k=0;k<d;++k){
      cd a=A[i*d+k];
      for(int j=0;j<d;++j)
        C[i*d+j]+=a*B[k*d+j];}
  return C;}
```

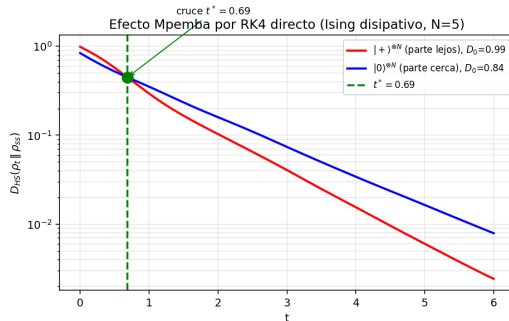
Paralelo

```
Mat matmul(A,B,d){
  Mat C(d*d);
  #pragma omp parallel for schedule(static)
  for(int i=0;i<d;++i)
    for(int k=0;k<d;++k){
      cd a=A[i*d+k];
      for(int j=0;j<d;++j)
        C[i*d+j]+=a*B[k*d+j];}
  return C;}
```

Una sola **#pragma omp parallel for** reparte las filas i entre hilos (independientes, sin condiciones de carrera). El *mismo* binario, con distinto OMP_NUM_THREADS, da las curvas serie vs paralelo.



Serie vs paralelo vs tamaño N : speedup $\rightarrow 4\times$.



Cruce de Mpemba por RK4: $t^* \approx 0.69$.

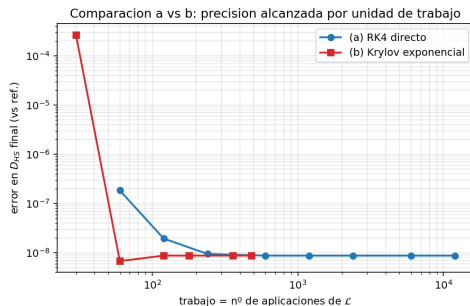
RK4 validado a orden 4 (≈ 4.04); speedup OpenMP sublineal (*bandwidth-bound*). **La curva que parte lejos adelanta a la cercana en t^* .**

4 · Evolución temporal (b): Krylov

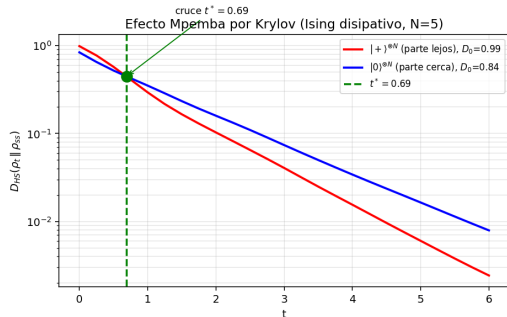
Sebastián

Idea. En vez de integrar paso a paso, aplicar la **acción del exponencial** $\rho(t+\tau) = e^{\tau\mathcal{L}}[\rho]$ proyectando en un subespacio de Krylov $\mathcal{K}_m$ (Arnoldi) y exponenciando la matriz pequeña $m \times m$ [19, 1].

- *Mismo hotspot* que RK4 (el `matmul` de $\mathcal{L}[\cdot]$)
 $\Rightarrow$ misma paralelización OpenMP, escalado equivalente.
- Ventaja: **mucho menos trabajo** para igual precisión, y pasos τ grandes (incondicionalmente estable).



A igual trabajo, Krylov $\sim 25\times$ más preciso que RK4.



Cruce en $t^* \approx 0.69$, idéntico al de RK4: el cruce es *físico*, no del integrador.

Krylov reproduce exactamente la misma trayectoria (acuerdo a 10^{-10} en D_{HS} final) con pasos de tiempo grandes.

5 · Muchos cuerpos: trayectorias (MPI)

Sebastián

Monte Carlo wavefunction [6, 9]: propagar M estados *puros* $|\psi\rangle \in \mathbb{C}^d$ (memoria d , no d^2) y promediar $\rho = \mathbb{E}[|\psi\rangle\langle\psi|]$. **Embarrassingly parallel**.

Serial

```
for(int tr=0;tr<M;++tr){
  psi=ket0;
  for(paso) RK4_Heff+salto(psi);
  acc += |psi><psi|;
}
rho = acc / M;
```

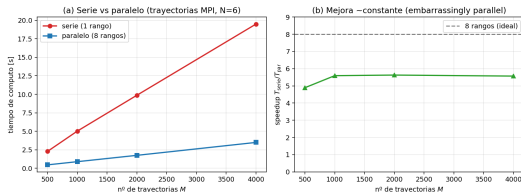
Paralelo (MPI)

```
int my_M = M/size + ...;    // reparto
for(int tr=0;tr<my_M;++tr){
  ... acc_local += |psi><psi|;
}
MPI_Reduce(acc_local,acc,MPI_SUM,0);
if(rank==0) rho = acc / M;   // 1 sola colectiva
```

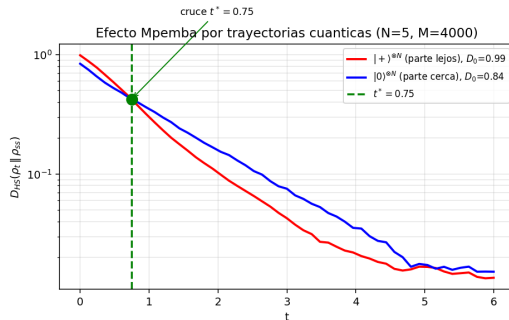
Cada rango integra su lote *sin comunicación*; una única **MPI_Reduce** promedia. Sin halo ni dependencias $\Rightarrow$ escalado casi ideal.

T3 — Resultados: computacional y físico

Sebastián

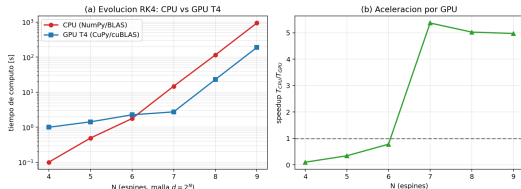


Serie vs paralelo vs M : speedup $\approx 5.6 \times \text{constante}$.



Cruce de Mpemba por trayectorias: $t^* \approx 0.75$.

Convergencia estadística $M^{-1/2}$; el MPI escala *casi ideal* (contraste con el OpenMP sublineal de T2):
el patrón de cómputo dicta el paradigma.



Evolución RK4: CPU vs GPU T4 (cuBLAS).

Tercer escalón: serie $\rightarrow$ OpenMP $\rightarrow$ **CUDA T4**. El matmul denso $\rightarrow$ cuBLAS zgemv.

- GPU pierde a d pequeño (overhead de kernels) y **gana** $\sim 5\times$ a $N \geq 7$ ($N=9$: 943 s CPU $\rightarrow$ 190 s GPU).
- GEMM hasta 250 GFLOP/s ($d=2048$).
- Validación triple GPU = CPU = C++ ($D_{HS} \rightarrow 0$).

Notebook ejecutado + resultados_t2_cuda.zip en
T2/cuda/.

6 · Muchos cuerpos: redes tensoriales

Juan

TEBD disipativo [21, 18]. La matriz densidad se escribe como un *superket* $|\rho\rangle\rangle$ (un MPS de dimensión física 4 por sitio) y se integra $\partial_t|\rho\rangle\rangle = \mathcal{L}|\rho\rangle\rangle$:

- **Trotter** de $e^{dt\mathcal{L}}$ en compuertas locales de 2 sitios.
- **Truncación** del rango de enlace a χ por SVD $\Rightarrow$ coste **polinómico en χ** (no exponencial en N).
- **Paralelización: hilos**. En una capa, los bonds *pares* (o impares) son disjuntos $\Rightarrow$ sus SVD ($\mathcal{O}(\chi^3)$) son independientes. numpy **libera el GIL** en la SVD/BLAS.

Serial

```
# capa de Trotter
for k in bonds:      # disjuntos
    mps.apply_gate(
        k, gate, chi) # SVD  $O(\chi^3)$ 
```

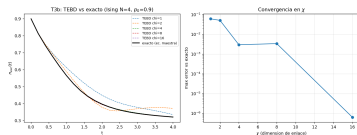
Paralelo (ThreadPool)

```
executor.map(
    lambda k: mps.apply_gate(k, gate, chi),
    bonds)      # SVD libera el GIL
# capas par / impar -> bonds independientes
```

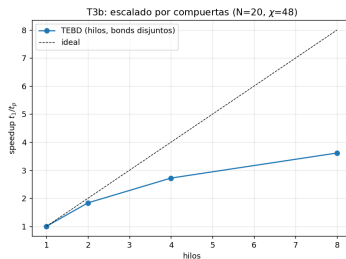
Paralelismo **estructurado**: intermedio entre el SpMV acoplado (T1) y las trayectorias independientes (T3). Se mapea a MPI repartiendo segmentos de la cadena.

T3b — Implementación y resultados

Juan



Error $\rightarrow 6 \times 10^{-7}$ al subir χ .

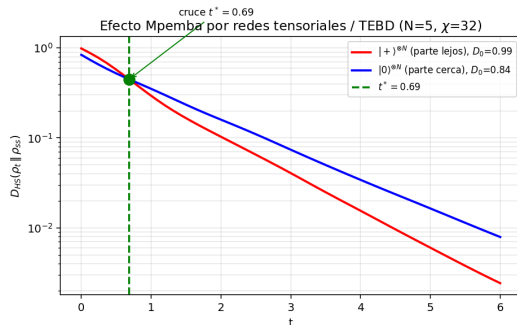


Hilos: $3.6 \times (8)$.

Alcance.

- Validado vs exacto: error cae con χ .
- $N = 32$ ($4^{32} \approx 10^{19}$), imposible en denso, en ~ 3.6 s.

Las redes tensoriales abren el régimen de muchos cuerpos donde T1/T2 no llegan.



También TEBD reproduce el efecto: $|+\rangle^{\otimes N}$ (parte lejos) **adelanta** a $|0\rangle^{\otimes N}$ en $t^* \approx 0.69$.

Idéntico a RK4/Krylov (a χ grande TEBD es exacto): **el cruce es físico**, y aquí se obtiene con el método que escala a muchos cuerpos.

Los **cuatro** métodos de evolución (RK4, Krylov, trayectorias, redes tensoriales) dan el mismo t^* .

7 · Conclusiones generales

Juan

HPC: el patrón de cómputo dicta el paradigma

- **T1** SpMV *acoplado* (OpenMP/MPI $\sim 2.9\times$).
- **T3b** *estructurado* (hilos $\sim 3.6\times$).
- **T3** *embarrassingly parallel* (MPI $\sim 5.6\times$, casi ideal).
- **T2** evolución densa $\rightarrow$ **GPU** (cuBLAS $\sim 5\times$).

Espectro de escalabilidad: acoplado $<$ estructurado $<$ independiente.

Física: el efecto Mpemba cuántico

- Origen *geométrico-espectral*: no-normalidad de $\mathcal{L}$ + criterio $a_2 = 0$ [4].
- **Cruce reproducido por los 4 métodos** de evolución (RK4, Krylov, trayectorias, redes tensoriales; $t^* \approx 0.69\text{--}0.75$), validado de forma cruzada.
- Escalable a muchos cuerpos vía trayectorias y redes tensoriales.

8 · Relevancia en el efecto macro/meso

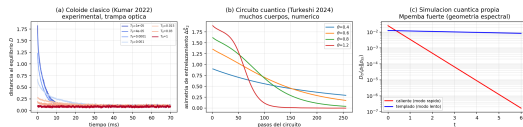
Sebastián

La firma es universal. El mismo mecanismo —**geometría espectral de la relajación**— explica el cruce en cascadas clásicas, coloides y sistemas cuánticos [13, 10, 20].

Lo que devuelve el enfoque cuántico:

- Un *criterio operacional* ($a_2 = 0$) y herramientas HPC para *predecir* y *diseñar* el efecto.
- Aplicaciones: **termometría cuántica** y enfriamiento acelerado [14, 8].

La firma universal del efecto Mpemba: cruce de curvas de relajación, del macro clásico al cuántico



La firma universal del cruce: del macro al cuántico.

Referencias I



A. H. Al-Mohy and N. J. Higham.

Computing the action of the matrix exponential, with an application to exponential integrators.
SIAM Journal on Scientific Computing, 33(2):488–511, 2011.



J. Bechhoefer, A. Kumar, and R. Chetrite.

A fresh understanding of the Mpemba effect.
Nature Reviews Physics, 3:534–535, 2021.



H.-P. Breuer and F. Petruccione.

The Theory of Open Quantum Systems.
Oxford University Press, 2002.



F. Carollo, A. Lasanta, and I. Lesanovsky.

Exponentially accelerated approach to stationarity in Markovian open quantum systems through the Mpemba effect.
Physical Review Letters, 127:060401, 2021.



P. Chattopadhyay, J. F. G. Santos, and A. Misra.

Anomaly to resource: the Mpemba effect in quantum thermometry.
arXiv preprint arXiv:2601.05046, 2026.



A. J. Daley.

Quantum trajectories and open many-body quantum systems.
Advances in Physics, 63(2):77–149, 2014.

Referencias II

- 
- V. Gorini, A. Kossakowski, and E. C. G. Sudarshan.
Completely positive dynamical semigroups of N-level systems.
Journal of Mathematical Physics, 17(5):821–825, 1976.
- 
- F. Ivander, N. Anto-Sztrikacs, and D. Segal.
Hyperacceleration of quantum thermalization dynamics by bypassing long-lived coherences.
Physical Review E, 108:014130, 2023.
- 
- J. R. Johansson, P. D. Nation, and F. Nori.
QuTiP 2: a Python framework for the dynamics of open quantum systems.
Computer Physics Communications, 184(4):1234–1240, 2013.
- 
- A. Kumar, R. Chétrite, and J. Bechhoefer.
Anomalous heating in a colloidal system.
Proceedings of the National Academy of Sciences, 119(5):e2118484119, 2022.
- 
- R. B. Lehoucq, D. C. Sorensen, and C. Yang.
ARPACK users' guide: Solution of large-scale eigenvalue problems with implicitly restarted Arnoldi methods.
SIAM, 1998.
- 
- G. Lindblad.
On the generators of quantum dynamical semigroups.
Communications in Mathematical Physics, 48(2):119–130, 1976.

Referencias III



Z. Lu and O. Raz.

Nonequilibrium thermodynamics of the Markovian Mpemba effect and its inverse.
Proceedings of the National Academy of Sciences, 114(20):5083–5088, 2017.



S. K. Manikandan.

Equidistant quenches in few-level quantum systems.
Physical Review Research, 3:043108, 2021.



F. Minganti, A. Biella, N. Bartolo, and C. Ciuti.

Spectral theory of Liouvillians for dissipative phase transitions.
Physical Review A, 98:042118, 2018.



E. B. Mpemba and D. G. Osborne.

Cool?
Physics Education, 4(3):172–175, 1969.



A. Nava and R. Egger.

Mpemba effects in open nonequilibrium quantum systems.
Physical Review Letters, 133:136302, 2024.



R. Orús.

A practical introduction to tensor networks: matrix product states and projected entangled pair states.
Annals of Physics, 349:117–158, 2014.

Referencias IV



R. B. Sidje.

Expokit: a software package for computing matrix exponentials.

ACM Transactions on Mathematical Software, 24(1):130–156, 1998.



G. Teza, J. Bechhoefer, A. Lasanta, O. Raz, and M. Vucelja.

Speedups in nonequilibrium thermal relaxation: Mpemba and related effects.

Physics Reports, 1164:1–97, 2026.



H. Weimer, A. Kshetrimayum, and R. Orús.

Simulation methods for open quantum many-body systems.

Reviews of Modern Physics, 93:015008, 2021.

¡Gracias!

Juan | Sebastián — Efecto Mpemba cuántico en HPC